화요일 학생 과제

기준 커밋: 0a5c603ff9ed432a47e92958a745931dc7c1225f. START 외 명령의 전송 전 정지 대기와 요청 해제까지 DO0 유지 규칙을 반영했다.

수정 파일은 학생코드/picknplace_full/sequence.py다. 통신부는 제공한다. AI 사용은 가능하며, 생성된 답을 실행하기 전에 요청·관측·완료 조건을 자신의 말로 정리한다. 문법 암기나 긴 코드 필사는 목표가 아니다. 아래 과제 문장과 실행 힌트는 구분해서 읽는다.

A — 명령의 결과

_accept에서 START 외 명령은 새 정지 관측을 기다린 뒤 한 번 보내고 접수일 때만 계속하며 대기 실패·거부·미확인은 단계와 이유를 남기고 중단하라.

실행 힌트: START(1111)는 전송 전 정지 대기를 생략한다. 나머지는 제공 _until에 robot == 1 조건과 move_timeout을 넘긴다. 이 대기가 실패하면 _send로 진행하지 않는다. _send(code)의 반환에는 result, code, reason이 있다. Abort는 자동 순서를 중단할 이유를 상위 함수에 전달하는 예외다. _result_text와 self.step은 제공 부분이다. 쓰기 호출이 끝났다는 사실과 PLC 접수 판정은 다르다.

검사: python -B -X utf8 check_student.py --stage A

제출: 세 판정과 전송 전 정지 대기의 계속/중단 여부를 쓰고 검사 결과를 첨부한다.

B-1 — 정상 순서

_cycle에서 자동 운전을 시작하고 이동1과 그리퍼 닫힘의 완료를 각각 확인한 뒤 이동2와 그리퍼 열림을 확인해 한 사이클을 완료하라.

실행 힌트: 요청은 _accept, 완료 조건은 _until에 넘긴다. 상태의 robot == 1은 정지, gripper == 1은 닫힘, gripper == 0은 열림이다. 제공 _until은 호출 이후의 새 관측을 기다린다. 자동 운전 중 그리퍼 전환은 PLC가 담당한다. 개별 제어용 3333을 자동 순서에 중복해서 넣지 않는다. 제한 시간은 move_timeout과 grip_timeout을 구분한다.

각 구간 앞에서 _set_step으로 UI 단계를 표시한다. 제공 화면과 시연에서 쓰는 이름은 1단계 자동 운전 시작, 2단계 이동1, 3단계 그리퍼 닫힘 기다림, 4단계 이동2, 5단계 그리퍼 열림 기다림이다. 함수 검사와 함께 실제 화면에 단계가 바뀌는지도 확인한다.

검사: python -B -X utf8 check_student.py --stage B --part normal

적용: A와 B-1 검사 통과 후 내부 모형에서 정상 경로를 확인하고, 가상장치실습안내대로 같은 학생코드 폴더의 세 프로그램을 실행한다. PC 대상 tcp:127.0.0.1:5020을 확인해 자기 sequence.py가 연결됐음을 기록한다. 로그에서 요청 세 개와 완료 대기 네 개를 구분한다. 접수만 받고 다음 동작으로 넘어가면 어느 충돌이 생길지 설명한다.

B-2 — 정지 경계

_cycle에서 정지 요청을 첫 시작 접수 뒤와 그리퍼 닫힘 확인 뒤에 처리하고 이동2를 보낸 뒤에는 현재 사이클을 끝내도록 보완하라.

실행 힌트: stop_requested는 요청을 기억하는 PC 변수이며 로봇 정지 신호가 아니다. _stop()은 제공된 자동 운전 해제 절차다. 정상 순서를 복사해서 별도 함수를 만들지 않고 같은 _cycle에 경계를 넣는다.

검사: python -B -X utf8 check_student.py --stage B --part stop

정지 요청 시점	예상 다음 동작	실행 근거 기록
1111 접수 뒤, 이동1 전	학생 작성	학생 작성
이동1 중	학생 작성	학생 작성
이동2 시작 뒤	학생 작성	학생 작성

C — 새 관측과 시간 제한

_until에서 새 관측을 얻지 못해 제한 시간을 넘기면 단계와 제한 시간을 이유로 남기고 후속 명령을 중단하라.

실행 힌트: 정상 await self.link.wait(pred, timeout)는 제공되어 있다. 반환된 None은 조건을 만족한 새 관측을 얻지 못했다는 뜻이다. 오래된 link.status를 읽어 완료라고 바꾸지 않는다. 화면에 보이는 마지막 정상 상태와 현재 통신 성공 여부는 별개다.

검사: python -B -X utf8 check_student.py --stage C

적용: lesson_scenarios.py --case read_fail과 --case read_timeout을 비교한다. 앞 사례는 짧은 실패 후 회복, 뒤 사례는 자동 순서의 대기 제한을 넘기는 조건이다. 뒤 사례는 A/B/C 완성 후 실행한다. writes에 다음 이동2가 없는지 확인하고, 통신 회복이 작업 완료를 뜻하지 않는 이유를 적는다.

D — 복구 전 개별 버튼

_why_off에서 자동 운전 여부가 미확정이면 개별 이동1·이동2·그리퍼 버튼을 보류하고 해제 접수 뒤 정상 상태에서 다시 허용하라.

실행 힌트: auto_maybe_on은 PC의 복구 필요 추정이다. 원본 정책은 이때 개별 세 버튼을 허용하며, 이번 과제는 교육 사본의 정책을 강화하는 변경이다. 통신·유효 상태 검사는 유지한다. 즉시 정지·자동 운전 해제·조건에 맞는 재개 경로를 일괄 차단하지 않는다. 버튼의 꺼진 이유를 빈 문자열이 아닌 문장으로 돌려주면 제공 UI와 실제 press가 함께 적용한다.

검사: python -B -X utf8 check_student.py --stage D

적용: 코드실행안내.md의 같은 UI 복구 절차에 따라 수정 전 허용, 수정 후 보류, 9994 접수 후 허용을 기록한다. 앱 재실행으로 초기화된 사실을 복구 성공으로 쓰지 않는다.

통합 재시험과 제출

정상·정지 경계·즉시 정지와 재개·읽기 실패와 시간 초과·쓰기 응답 유실·복구 후 재실행을 시험하고 각 결과를 명령과 최신 관측 근거로 설명하라.

사례	예상	실제 결과·로그 시각	다를 때 수정	재시험
정상 한 사이클	학생 작성	학생 작성	학생 작성	학생 작성

이동1 중 정지 요청	학생 작성	학생 작성	학생 작성	학생 작성
이동2 후 정지 요청	학생 작성	학생 작성	학생 작성	학생 작성
즉시 정지 후 재개	학생 작성	학생 작성	학생 작성	학생 작성
읽기 실패 후 회복	학생 작성	학생 작성	학생 작성	학생 작성
제한 시간을 넘긴 새 관측 대기	학생 작성	학생 작성	학생 작성	학생 작성
쓰기 응답만 유실	학생 작성	학생 작성	학생 작성	학생 작성
9994 해제 후 다음 실행	학생 작성	학생 작성	학생 작성	학생 작성

수정한 sequence.py, 전체 검사 결과, 위 표를 제출한다. 각 줄에 내부 모형/세 프로그램 가상장치/실물 중 무엇을 시험했는지 적는다. 가상장치만 했다면 실물 미검증이라고 적는다.

자기 프로그램과 가상장치 관측 과제

가상장치실습안내를 따라 자기 학생코드에서 세 프로그램을 실행하라. 다음 기록을 실습기록지에 남겨라.

1. PLC RUN 뒤 준비 타이머·M00418과 PC 상태를 대조하고, 0002만으로 시작 접수가 보장되지 않는 이유를 적어라.
2. 이동1 요청부터 로봇 DI0/DO0·PLC P00044/P00004·D01000·PC 새 관측까지 연결해 기록하라. 이동1 중 즉시 정지는 로봇 DO0 ON·실제 움직임과 PLC RUN1(M00414) ON을 함께 확인한 뒤 시험하라. 이동2에서는 RUN2(M00415)를 확인하라. 이 비트는 PLC가 로봇 시작을 확인해 기억하는 내부 값이며 PLC 운전 모드 RUN과 다르다.
3. 3단계 닫힘 중 즉시 정지에서는 닫힘 완료 후에도 9992가 보류되는지, 재개 뒤 나가는지 확인하라. 5단계 열림 중 즉시 정지에서는 재개 없이 작업이 완료되어도 robot=2가 남는지 확인하라. 두 결과가 다른 이유를 다음 명령의 유무로 설명하라.
4. 로봇을 제공 robot/PICKNPLACE_FULL_main.js 파일 모드로 실행하고 선택한 파일명을 기록하라. 파일을 수정했다면 다시 선택하라. PC 파일 수정은 PC 앱 재실행으로 적용하라.
5. 예상과 다른 조건 하나의 마지막 정상 관측과 후속 요청을 찾고, 한 가지 수정 후 같은 조건으로 재시험하라. 초기화로 새 시험을 만든 경우를 같은 작업의 복구라고 쓰지 마라.

파일 모드의 ES5 문법 제한은 가짜 실행기의 제한이다. 로봇 파일의 Thread를 새로 구현하거나 예시 좌표를 실물에 적용하는 과제는 아니다.